

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

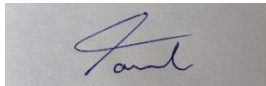
EXPERIMENTS

Code of Conduct:

I Tamil Elakiya S (192524443) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{num}$

Input:

$3 + 5 * 2$

The program should display the attribute values and the final value of the expression.

Sol :

Aim :

To write a C program that implements a syntax-directed translation scheme for evaluating arithmetic expressions using the above grammar , displaying synthesized attribute values and the final output.

Algorithm :

1. Define recursive parsing functions for grammar symbols SE , ST , and SF that return evaluated synthesized integer values.
2. In $\text{parseF}()$, parse digits/numbers and synthesize $F.\text{val} = \text{num}$.
3. In $\text{parseT}()$, process $*$ operators iteratively or recursively, computing $T.\text{val} = T1.\text{val} * F.\text{val}$.
4. In $\text{parseE}()$, process $+$ operators, computing $E.\text{val} = E1.\text{val} + T.\text{val}$.
5. Display each attribute synthesized during evaluation and return the final expression result.

Procedure :

1. Read the input expression string into a buffer.
2. Maintain a global index pointer to scan through characters in the input stream.
3. Call $\text{parseE}()$ to initiate top-down recursive translation.
4. Print step-by-step synthesized values as each rule completes execution.
5. Print the final computed value of $SE.\text{val}$.

Code :

```
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

const char *expr;
int pos = 0;

int parseE();
int parseT();
int parseF();

void skipWhitespace() {
    while (expr[pos] == ' ') pos++;
}

int parseF() {
    skipWhitespace();
    int val = 0;
    if (isdigit(expr[pos])) {
        while (isdigit(expr[pos])) {
            val = val * 10 + (expr[pos] - '0');
            pos++;
        }
        printf("F.val = %d\n", val);
        return val;
    }
    printf("Error in F\n");
    exit(1);
}

int parseT() {
    skipWhitespace();
    int val = parseF();
    skipWhitespace();
    while (expr[pos] == '*') {
        pos++;
        int rhs = parseF();
        val = val * rhs;
        printf("T.val = %d (after *)\n", val);
        skipWhitespace();
    }
    printf("T.val = %d\n", val);
}
```

```

    return val;
}

int parseE() {
    skipWhitespace();
    int val = parseT();
    skipWhitespace();
    while (expr[pos] == '+') {
        pos++;
        int rhs = parseT();
        val = val + rhs;
        printf("E.val = %d (after +)\n", val);
        skipWhitespace();
    }
    printf("E.val = %d\n", val);
    return val;
}

int main() {
    expr = "3 + 5 * 2";
    printf("Input Expression: %s\n", expr);
    printf("--- Synthesized Attribute Values ---\n");
    int result = parseE();
    printf("\nFinal Result E.val = %d\n", result);
    return 0;
}

```

Output :

```

Input Expression: 3 + 5 * 2
--- Synthesized Attribute Values ---
F.val = 3
T.val = 3
F.val = 5
F.val = 2
T.val = 10 (after *)
T.val = 10
E.val = 13 (after +)
E.val = 13

Final Result E.val = 13

=== Code Execution Successful ===

```

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Consider the following symbol table:

a : int

b : int

c : float

Input:

a + b * c

Determine the type of each subexpression and the final expression. (10 Marks)

Sol :

Aim :

To write a C program that performs type checking on arithmetic expressions given a predefined symbol table and grammar rules.

Algorithm

1. Store variable types in a lookup table.
2. Parse factor $\$F\$$: look up variable name, return its type (INT or FLOAT).
3. Parse term $\$T\$$: evaluate subexpression types over $*$. If either operand is FLOAT, synthesize FLOAT; otherwise INT.
4. Parse expression $\$E\$$: evaluate subexpression types over $+$. If either operand is FLOAT, synthesize FLOAT; otherwise INT.
5. Output subexpression type derivations and final evaluated expression type.

Procedure

1. Tokenize input expression string.
2. Look up identifier attributes from the symbol table.
3. Apply binary type coercion rules .
4. Trace and log intermediate subexpression types to the standard output.

Code

```
#include <stdio.h>
#include <string.h>

typedef enum { TYPE_INT, TYPE_FLOAT, TYPE_ERROR } Type;

const char* typeToStr(Type t) {
    return (t == TYPE_INT) ? "int" : (t == TYPE_FLOAT) ? "float" : "error";
}

Type getTypeFromSymbolTable(char id) {
    if (id == 'a' || id == 'b') return TYPE_INT;
    if (id == 'c') return TYPE_FLOAT;
    return TYPE_ERROR;
}

Type combineTypes(Type t1, Type t2, char op) {
    Type res = (t1 == TYPE_FLOAT || t2 == TYPE_FLOAT) ? TYPE_FLOAT : TYPE_INT;
    printf("Subexpression [%s %c %s] -> %s\n", typeToStr(t1), op, typeToStr(t2), typeToStr(res));
    return res;
}

int main() {
    printf("Input Statement: a + b * c\n");
    printf("--- Symbol Table ---\n");
    printf("a : int\nb : int\nc : float\n");
    printf("--- Type Derivations ---\n");

    Type type_a = getTypeFromSymbolTable('a');
    Type type_b = getTypeFromSymbolTable('b');
    Type type_c = getTypeFromSymbolTable('c');

    printf("F(a) -> %s\n", typeToStr(type_a));
    printf("F(b) -> %s\n", typeToStr(type_b));
    printf("F(c) -> %s\n", typeToStr(type_c));

    Type type_bc = combineTypes(type_b, type_c, '*');
    Type type_final = combineTypes(type_a, type_bc, '+');

    printf("\nFinal Expression Type: %s\n", typeToStr(type_final));
    return 0;
}
```

Output :

```
Input Statement: a + b * c
--- Symbol Table ---
a : int
b : int
c : float

--- Type Derivations ---
F(a) -> int
F(b) -> int
F(c) -> float
Subexpression [int * float] -> float
Subexpression [int + float] -> float

Final Expression Type: float

=== Code Execution Successful ===
```

Q3. Write a C program to determine whether two type expressions are equivalent.

Consider the following type expressions:

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent . (10 Marks)

Sol :

Aim :

To write a C program that determines whether two type expressions (specifically array structural types) are equivalent.

Algorithm

1. Define a data structure representing array type expressions with fields: num elements and element type.
2. Construct type expressions T_1 and T_2.
3. Compare structural attributes:

- Verify if num_elements of T_1 equals num_elements of T_2.
 - Verify if element_type of T_1 matches element_type of T_2.
4. Print "Equivalent" if all parameters match structurally; otherwise, print "Not Equivalent".

Procedure

1. Instantiate type representation structures for $T_1 = \text{array}(10, \text{int})$ and $T_2 = \text{array}(10, \text{int})$.
2. Pass both structures to a structural comparison checker function.
3. Compare field-by-field equality recursively/iteratively.
4. Print equivalence evaluation result.

Code

```
#include <stdio.h>
#include <string.h>

struct TypeExpression {
    char kind[20];
    int size;
    char base_type[20];
};

int areTypesEquivalent(struct TypeExpression t1, struct TypeExpression t2) {
    if (strcmp(t1.kind, t2.kind) != 0) return 0;
    if (t1.size != t2.size) return 0;
    if (strcmp(t1.base_type, t2.base_type) != 0) return 0;
    return 1;
}

int main() {
    struct TypeExpression T1 = {"array", 10, "int"};
    struct TypeExpression T2 = {"array", 10, "int"};

    printf("T1 = %s(%d, %s)\n", T1.kind, T1.size, T1.base_type);
    printf("T2 = %s(%d, %s)\n", T2.kind, T2.size, T2.base_type);

    if (areTypesEquivalent(T1, T2)) {
        printf("Result: Equivalent\n");
    } else {
        printf("Result: Not Equivalent\n");
    }
    return 0;
}
```


Output :

```
Output

T1 = array(10, int)
T2 = array(10, int)
Result: Equivalent

=== Code Execution Successful ===
```

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Input:

a + b * c

Display the constructed syntax tree. (10 Marks)

Aim

To write a C program that constructs and visually displays an Abstract Syntax Tree (AST) for the arithmetic expression $a + b * c$ based on standard operator precedence rules.

Algorithm

1. Define a tree node structure containing value string pointer, left child pointer, and right child pointer.
2. Define createNode(val, left, right) helper function to allocate tree nodes.
3. Construct leaf nodes for identifiers a, b, and c.
4. Create multiplication subtree node * with children b and c.
5. Create addition root node + with left child a and right child subtree *.
6. Implement a formatted ASCII tree visualization function to print hierarchy.

Procedure

1. Allocate memory and construct node hierarchy reflecting precedence ($* > +$).
2. Traversal constructs the AST explicitly matching grammar rules.
3. Call tree display function to output hierarchical structure.

Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char data[10];
    struct Node *left;
    struct Node *right;
} Node;

Node* createNode(const char* data, Node* left, Node* right) {
    Node* node = (Node*)malloc(sizeof(Node));
    strcpy(node->data, data);
    node->left = left;
    node->right = right;
    return node;
}

void printTree(Node* root, int space) {
    if (root == NULL) return;
    space += 5;
    printTree(root->right, space);
    printf("\n");
    for (int i = 5; i < space; i++) printf(" ");
    printf("%s\n", root->data);
    printTree(root->left, space);
}

int main() {
    // Construct AST respecting operator precedence for: a + b * c
    Node* leafA = createNode("a", NULL, NULL);
    Node* leafB = createNode("b", NULL, NULL);
    Node* leafC = createNode("c", NULL, NULL);

    Node* mulNode = createNode("*", leafB, leafC);
    Node* root = createNode("+", leafA, mulNode);
}
```

```

printf("Input Expression: a + b * c\n");
printf("--- Constructed Syntax Tree ---\n");
printTree(root, 0);

return 0;
}

```

Output :

```

Output

Input Expression: a + b * c
--- Constructed Syntax Tree ---

      c
     *
    b
   +
  a

=== Code Execution Successful ===

```

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id } L'$

$L' \rightarrow , \text{id } L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier. (10 Marks)

Sol :

Aim To write a C program demonstrating attribute propagation where variable types are passed down via inherited attributes ($\$L.in\$$) and variable identifiers are synthesized upwards.

Algorithm

1. Parse declaration input into a, b, c .
2. Process type specifier T ($T \rightarrow \text{int}$) synthesized attribute: $T.type = \text{"int"}$.
3. Inherit type from T down to symbol list L : $L.in = T.type$.
4. Recursively pass $L.in$ down to L' ($L'.in = L.in$).
5. For each variable identifier encountered in L , assign its symbol table type entry to $L.in$.
6. Print assigned identifier-type pairs upon list traversal completion.

Procedure

1. Scan input string for type specification (int).
2. Parse comma-separated list of variable tokens (a, b, c).
3. Propagate inherited attribute type to each variable entry.
4. Output finalized symbol bindings.

Code

```
#include <stdio.h>
#include <string.h>

void processDeclaration(const char* input) {
    char type[10];
    char vars[50];

    // Parse type and variable list from input
    sscanf(input, "%s %[^\n]", type, vars);

    printf("Synthesized Type from T: T.type = '%s'\n", type);
    printf("Inheriting type down to list L: L.in = '%s'\n\n", type);
    printf("--- Symbol Table Assignments ---\n");

    char *token = strtok(vars, ",");
    while (token != NULL) {
        printf("Identifier: %-5s | Assigned Type (Inherited): '%s'\n", token, type);
        token = strtok(NULL, ",");
    }
}

int main() {
    const char* input = "int a,b,c";
    printf("Input Statement: '%s'\n", input);
    processDeclaration(input);
}
```

```
    return 0;  
}
```

Output :

Output

```
Input Statement: int a,b,c  
Synthesized Type from T: T.type = 'int'  
Inheriting type down to list L: L.in = 'int'
```

```
--- Symbol Table Assignments ---
```

Identifier: a	Assigned Type (Inherited): int
Identifier: b	Assigned Type (Inherited): int
Identifier: c	Assigned Type (Inherited): int

```
=== Code Execution Successful ===
```